

Making Software Sustainability Actionable: Metrics, Patterns, and Processes for Resource-Sensitive Development

Research-in-Progress

Tobias Walter ¹, Anett Mehler-Bicher ¹, Jan Conrad ², and Dieter Wallach ²

Abstract: Decisions in software development influence computational effort, data transfer, storage demand, and hardware utilization. Consequently, sustainability should not be treated as an external concern, but as an integral part of the software development process. This research in progress paper presents an approach for developing artifacts that support resource-sensitive software development. The proposed approach combines a sustainable software development process, a role concept for the resource-sensitive developer, resource-sensitive software patterns, and sustainability metrics. These artifacts aim to make resource consumption visible, comparable, and actionable across the software lifecycle.

Keywords: Sustainability, Software Development, Resource-Sensitive Metrics, Patterns

1 Introduction

Digital technology is one of the major contributors to global greenhouse gas emissions, accounting for more than 4% of the global CO₂ footprint annually [Ax26]. Global IT energy use could exceed 20% of total energy and reaching up to 5.5% of the world's carbon emissions by 2025 [Gu24]. The increasing use of cloud computing and energy-intensive artificial intelligence is expected to further intensify the demand for computational resources. As software systems continue to shape business processes, public services, and everyday life, their design, development, operation, and maintenance become increasingly relevant from a sustainability perspective. Software development is understood as a problem-solving process in which business requirements, user needs, technical constraints, economic conditions, and sustainability aspects must be balanced. Sustainability also plays a decisive commercial role: according to [Ca23], 79% of consumers consider sustainability criteria in their purchasing decisions. Sustainability is therefore not treated as an external add-on, but as an essential boundary condition for contemporary software development. Analogous to the curb-cut effect described by [An17], this research assumes that the systematic consideration of

¹ Mainz University of Applied Sciences , Lucy-Hillebrand-Straße 2, 55128 Mainz, Germany, tobias.walter@hs-mainz.de,  <https://orcid.org/0000-0001-8430-0235>; anett.bicher@hs-mainz.de,  <https://orcid.org/0000-0003-1577-2276>

² Kaiserslautern University of Applied Sciences , Amerikastraße 1, 66482 Zweibrücken, Germany, jan.conrad@hs-kl.de,  <https://orcid.org/0000-0002-0222-8856>; dieter.wallach@hs-kl.de,  <https://orcid.org/0000-0002-8882-9512>

sustainability aspects can create benefits beyond the goal of reducing resource consumption. Actions introduced to support resource-sensitive software development may improve transparency and stakeholder trust [Kh20]. Resource efficiency refers to the ratio of useful work performed to the resources consumed [Gr18]. Resource-efficient software development can be operationalised as the successful execution of standard usage scenarios for software development.

Resource-Sensitive Software Development Process and Roles. Although the importance of sustainability in software development is widely acknowledged, practical methods for integrating sustainability concerns into the software development process are not yet sufficiently established. The resource consumption of software systems is influenced by decisions throughout the entire lifecycle of software development, including requirements engineering, architecture and design, implementation, testing, deployment, operation, and maintenance. Our research in progress therefore investigates how existing software development processes can be extended through the continuous inclusion of resource-sensitivity considerations. The research also addresses the role of the resource-sensitive software developer. Software teams need concrete guidance, competencies, and responsibilities that enable them to consider sustainability in design and implementation decisions.

Metrics for Measuring Resource Sensitivity. A further focus concerns metrics. Without appropriate metrics, software resource consumption often remains difficult to manage. Metrics can create transparency regarding the resource-related effects of software decisions and may support trade-off analyses between functionality, usability, and sustainability. In this research, sustainability metrics are therefore considered an essential element for making resource consumption observable, comparable, and actionable within the software development process.

Resource-Sensitive Software Patterns. In addition, the project focuses on resource-sensitive software patterns. Such patterns capture reusable solutions to recurring sustainability-related design problems and help requirements engineers, developers, and architects make resource-sensitive decisions. Our research aims to identify, analyze, extend, and document software patterns that support the development of resource-sensitive software systems.

Application Contexts. The proposed process extensions, role concepts, patterns, and metrics will be explored and evaluated through selected case studies. A first case study addresses the resource-sensitive use of large language models, where energy demand and computational costs are becoming increasingly relevant. A second case study focuses on mobile and web design, where lightweight design and development decisions can reduce data transfer, server load, and client-side computational effort while decreasing resource consumption. A third case study considers resource-sensitive development by citizen developers working on low-code/no-code platforms.

2 Related Work

Sustainable software has gained increasing relevance. Existing work on sustainable software products, resource-sensitive software development processes, roles, and software patterns provides an important foundation for this research.

Sustainable Software Products. Assessment criteria for resource and energy efficiency show how software products can be evaluated with regard to their hardware demand, energy consumption, and resource use [Ke18]. The Green Software Measurement Model further advances this perspective by proposing a generic reference model for measuring the energy and resource efficiency of software products and components. These contributions are highly relevant because they define conceptual and methodological foundations for making software resource consumption measurable [Na11]. However, their primary focus lies on the assessment and measurement of software products or components. The present research therefore builds on this work. It investigates how sustainability metrics can be embedded into software development processes to support the implementation of resource-sensitive software products. This enables development decisions to be guided before the resource consumption of software products is measured retrospectively. The Green Software Measurement Model (GSMM) by [Gu24] provides an important reference model for assessing the energy and resource efficiency of software products and components. Our research builds on this measurement-oriented perspective, but differs in its primary objective. Rather than developing another reference measurement model, we focus on integrating sustainability metrics into the software development lifecycle as action-guiding artifacts. In this way, sustainability is addressed not only as an ex-post assessment of software products or components, but as a continuous design and development concern.

Resource-Sensitive Software Development Processes and Roles. A central prerequisite for sustainable software is a sustainable software development process. Existing approaches provide initial starting points. For example, the concept of the non-human persona extends human-centered design by representing nature as a non-human stakeholder in design processes [To21]. Such artifacts can help broaden the perspective of software teams beyond immediate user and business requirements. However, they remain only partially integrated into overarching software development processes. Similarly, established approaches such as user experience metrics define target and acceptance values for relevant aspects of user experience at the beginning of a project [SW22; WCS17]. This suggests a transferable idea for sustainability-oriented software development: for identified user journeys and relevant usage scenarios, sustainability metrics could be defined as benchmarks that guide subsequent design and development work. Further related work concerns the measurement and quantification of sustainability. Without appropriate metrics, the resource consumption of software remains difficult to observe, compare, and manage. Sustainability metrics can create transparency regarding energy consumption, CO₂ footprint, data transfer, computational effort, storage demand, and efficiency indicators for frontend, backend, infrastructure, and AI-based systems. Such metrics are necessary to make sustainability actionable within software development processes. They are also required to evaluate resource-sensitive

software patterns and to provide evidence for their practical relevance. This gap is also relevant for the developer role: software developers need not only technical implementation skills, but also competencies for assessing resource implications, selecting appropriate patterns, and integrating sustainability concerns into design and architecture decisions.

Resource-Sensitive Software Patterns. Furthermore, the proposed approach connects process, metrics, and roles with resource-sensitive software patterns. The use of software patterns, as shown in [St25], represents another relevant foundation. Patterns are established solution templates for recurring design problems and have been applied successfully in software development, human-computer interaction, mobile and web design contexts [PKM10]. In the context of resource-sensitive software development, software patterns can provide practical guidance for developers when making design and implementation decisions. Examples include patterns for resource-efficient infrastructure choices, such as on-premises, cloud, or serverless approaches, as well as patterns that address API calls, database queries, storage demand, data transfer, or energy consumption. Resource-sensitive design and development patterns are currently fragmented. Although several approaches address sustainable design and development practices [An23; Ba23; Gr21; Sc26], there is still a lack of a central repository that documents relevant patterns, illustrates their practical application through case studies, and provides indicators for their expected resource impact [An23].

Synthesis. Taken together, related work indicates that sustainable software development requires the integration of four perspectives: process integration, design patterns, roles, and measurable indicators. Human-centered software development provides methodological foundations for structured requirements work and measurable quality goals [SW22; WCS17]. Pattern-based approaches provide reusable design knowledge [PKM10]. Sustainability metrics can make resource consumption visible and comparable. However, these strands are not yet sufficiently combined into an integrated approach that supports software teams across the entire software development lifecycle.

3 Research Problem and Objectives

The central research problem addressed in this paper is the lack of an integrated approach for embedding sustainability considerations into software development processes. Although sustainability is increasingly recognized as a relevant quality and decision criterion, software teams often lack concrete artifacts, metrics, patterns, and role descriptions that support resource-sensitive decisions in everyday development practice. This research in progress investigates how sustainability metrics, resource-sensitive software patterns, and action-guiding process artifacts can be integrated into software development practice. Hence, we define the following objectives:

Objective 1 Sustainable Software Development Process. The first objective is to extend existing software development and human-centered processes with sustainability-oriented artifacts. In particular, the research explores how sustainability metrics can be integrated in

a way analogous to UX metrics tables, in which target and acceptance values are defined early in the project and used to guide design and development decisions [SW22], [WCS17]. This includes the analysis of established software development models, agile approaches, and human-centered process models in order to determine where sustainability-related activities, artifacts, and decision points can be embedded.

Objective 2 Role Concept. The second objective is to develop a role concept for the resource-sensitive software developer. The role concept clarifies the responsibilities, competencies, and tasks required to implement sustainability-oriented software development in practice, including resource-conserving requirements development, software architectures, quality assurance, performance monitoring, deployment, and operation strategies.

Objective 3 Resource-Sensitive Software Patterns. The third objective is to identify and document resource-sensitive software patterns. They are intended to provide reusable guidance for recurring sustainability-related design and development problems. They may address infrastructure decisions, API usage, database access, storage strategies, data transfer, user interface design, mobile and web optimization, and the resource-sensitive use of AI-based systems.

Objective 4 Sustainability Metrics. The fourth objective is to identify and structure sustainability metrics that enable a valid and reliable assessment of software-related resource consumption. These metrics are intended to create transparency regarding the environmental and resource-related effects of software systems. Relevant indicators include, for example, energy consumption, CO₂ footprint, data transfer, computational effort, storage demand, and efficiency indicators for sustainable frontend and backend design. The metrics should be applicable during the development process and support decision-making in relation to requirements, user journeys, architecture, implementation, and operation.

Based on these objectives, the guiding research question is:

How can software development be extended with metrics, patterns, and role concepts to support resource-sensitive software development across the entire software lifecycle?

This overarching question can be further specified through the following sub-questions:

RQ1 How can sustainability-oriented artifacts be integrated into existing software development process models?

RQ2 Which competencies, responsibilities, and learning paths characterize the role of a resource-sensitive software developer?

RQ3 Which resource-sensitive software patterns can support sustainable design and development decisions, and how can their resource implications be documented for practical use?

RQ4 Which sustainability metrics are suitable for making the resource consumption of software systems transparent, comparable, and actionable during the software development process?

Beyond the initial development and evaluation of the proposed artifacts, the research aims at long-term consolidation and practical transfer. The resulting resource-sensitive software patterns and sustainability metrics will be made available through an open repository to support reuse, transparency, and further refinement by the community. To facilitate simple and rapid application in practice, the project will also investigate how selected patterns can be integrated into low-code/no-code platforms for citizen developers. In addition, the role concept and associated competencies will inform curricular integration and continuing education offers for software professionals. The practical applicability of the approach will be explored through pilot deployments with industry partners. In the long term, the research also aims to provide a foundation for a sustainability label that certifies software development processes with regard to their systematic integration of resource-sensitive practices.

4 Methodology

This research follows a Design Science Research approach to develop and evaluate artifacts for resource-sensitive software development. The first artifact is a sustainable software development process that extends existing development models with action-guiding sustainability activities, a role concept for the resource-sensitive developer, and resource-sensitive software patterns. The second artifact consists of sustainability metrics that make software-related resource consumption observable, comparable, and actionable during the development lifecycle. These artifacts are iteratively designed, refined, and evaluated in the following application contexts.

Evaluation and Application Context: Mobile and Web Design. Sustainable web design is a relevant application context because both the number of internet users and the amount of transferred data continue to grow. Against this background, lightweight and energy-efficient web applications are of high relevance [W326]. [Da20] illustrates how seemingly small design decisions can reduce the annual CO₂ footprint. Lightweight websites can also improve loading times and user experience and may contribute to improved conversion rates [Gr21].

Evaluation and Application Context: On-Premise Use of Large Language Models. The increasing use of artificial intelligence and large language models adds another dimension to sustainable software development. The development of state-of-the-art large language models is associated with substantial costs for specialized hardware, energy, and qualified personnel. Current analyses expect training costs to potentially grow in the coming years [Co24; SWC25]. Recent discussions therefore call for a sustainable AI revolution and criticize the insufficient efficiency of current large language model training paradigms. Instead of measuring the resource consumption of data centers, which is difficult to assess in practice, our focus lies on the use of LLMs trained, tested, and used on-premises on local hardware.

Evaluation and Application Context: Low-Code/No-Code Platforms. Low-code/no-code platforms provide a relevant context for integrating sustainability metrics and resource-sensitive software patterns directly into development environments. Suitable metrics, such as energy consumption, CO₂ footprint, data transfer, and frontend efficiency indicators, will be identified and embedded as examples to make resource consumption transparent during application modeling. This is particularly relevant for citizen developers, who typically create applications with limited software development expertise and only minimal support from IT departments. By integrating selected resource-sensitive patterns as reusable building blocks, recommendations, or automated feedback, low-code/no-code platforms can support more sustainable design decisions without requiring detailed technical knowledge from users.

5 Preliminary Findings

The research is still in an early stage; therefore, the following findings represent preliminary concepts rather than finalized or evaluated results. They summarize artifacts and design decisions that currently structure the development of the proposed approach. The findings are organized along the core elements of the research:

Resource-Sensitive Software Development Process. Initial work indicates that sustainability can be operationalized as a dedicated quality dimension within the software development lifecycle if it is linked to explicit metrics, decision artifacts, and lifecycle-specific checkpoints that translate resource sensitivity into concrete development practices. These include green acceptance criteria for user stories, sustainability-related checklists for lifecycle activities, and a green definition of done that requires resource targets to be measured, documented, and protected against regressions before a feature or change is considered complete. A preliminary set of candidate metrics has been identified to support this operationalization. These metrics include energy per request or service call, watt-hours per merge request, CO₂ emissions per build or release, storage demand per transaction, and data transfer per service. In addition to product-related metrics, early process indicators appear relevant, such as the share of pull requests that include a green checklist, the number of architecture decision records documenting resource impacts, the coverage of energy tests in continuous integration pipelines, and the mean time to detect resource regressions. Another preliminary finding concerns the need to make sustainability-related trade-offs explicit, since resource-sensitive software development may conflict with time-to-market, performance goals, cost constraints, or usability requirements. The development approach therefore combines feature-level carbon budgeting with a trade-off matrix. For each relevant feature, a target corridor for resource consumption can be defined during backlog refinement or requirements development. Deviations from this corridor require an explicit architecture decision record, thereby making sustainability-related trade-offs transparent and accountable. Performance optimization is consequently framed not only in terms of speed or throughput, but also in terms of performance per watt or performance per cost unit. The initial process concept further suggests that sustainability should be embedded into existing governance

structures rather than introduced as a separate parallel process. Candidate integration points include the definition of done, architecture boards, quality gates, security reviews, and continuous integration pipelines. To support this integration, preliminary templates are being developed, including an extension of architecture decision records into green architecture decision records and a checklist-based green definition of done. These artifacts are intended to make sustainability considerations actionable for development teams while remaining compatible with established agile and software development practices.

Resource-Sensitive Software Patterns. A second preliminary result is the structure of a repository for resource-sensitive software patterns and anti-patterns. The current concept links each pattern to three additional elements: a responsible role, suitable measurement methods, and audit criteria. This creates traceability from the initial design recommendation to its implementation and verification, as each pattern should indicate who is responsible for its application, how its effect can be measured, and which evidence is required to demonstrate its correct use. Initial candidate patterns have been identified particularly for low-code/no-code environments. These include delta synchronization instead of full synchronization, batch actions instead of multiple individual service calls, event-triggered processing instead of polling, client-side caching, and automatic asset optimization, such as image resizing or format conversion. These patterns are intended to be offered as reusable building blocks that can be applied by citizen developers without requiring detailed technical knowledge of the underlying implementation. The preliminary pattern concept also includes an economic perspective. For each pattern, the repository should provide an indicative business case that relates expected resource savings to implementation effort and operational benefits, for example expected reductions in CPU usage, request latency, or infrastructure costs for a response-caching pattern. This is intended to support informed decision-making and to make trade-offs between resource savings, complexity, and maintainability more transparent. To avoid overburdening citizen developers, the patterns are planned to be supported by simplified interaction mechanisms such as traffic-light indicators, wizard-based configuration, automated fixes, and real-time design feedback. An energy label for patterns and a green score during application design could help users understand the resource implications of their choices while keeping the technical details manageable. In this way, resource-sensitive patterns may become directly actionable in development environments and low-code/no-code platforms.

Resource-Sensitive Developer. A third preliminary result concerns the definition of the resource-sensitive developer as a role-based bundle of competencies, responsibilities, and artifacts. The role is not understood as a general attitude toward sustainability, but as a process- and tool-embedded responsibility. Initial role artifacts include a RACI matrix for sustainability-related development tasks, resource acceptance criteria as part of the definition of done, green architecture decision records that document the resource impact of design decisions, energy observability in CI/CD pipelines, and feature-level carbon budgets. The resource-sensitive developer focuses on implementation-oriented optimization and on providing evidence for the resource impact of software decisions across requirements, design, code, and runtime. This includes making design and code decisions with measurable

resource implications, selecting appropriate resource-sensitive patterns, and contributing to the continuous monitoring of energy and resource indicators. The role thereby connects strategic sustainability goals with concrete development practice. The effectiveness of the role will be assessed through a combination of product-related and process-related indicators, including energy per request or service call, CO₂ emissions per build or release, storage demand per transaction, and data transfer per service. To distinguish individual developer contributions from broader system effects, the emerging approach combines before-and-after comparisons at the merge-request level, such as changes in watt-hours per request, with component-level carbon budgets. Adoption and organizational effectiveness are expected to be measured through additional process indicators, including the share of developers trained or certified for the role, the percentage of pull requests that include a green checklist, and the mean time to detect resource regressions. Together, these indicators show whether the role becomes operationally effective in development practice and whether sustainability responsibilities are consistently reflected in everyday development workflows.

6 Discussion and Next Steps

The proposed approach contributes to environmental and sustainability informatics by connecting sustainability metrics, resource-sensitive software patterns, and process-oriented artifacts for software development practice. Its practical relevance lies in making resource consumption more visible and actionable for development teams, citizen developers, and organizations aiming to integrate sustainability into everyday software development. From a scientific perspective, the work contributes to the operationalization of sustainable software development by linking measurement, environmental knowledge, and role-based responsibilities. As this is research in progress, the current artifacts are still preliminary and have not yet been comprehensively evaluated. Open questions concern the validity and comparability of selected metrics, the transferability of patterns across different application contexts, and the adoption of the proposed role concept in practice. Next steps include refining the process model, documenting the pattern and metrics repository, and evaluating the approach through case studies on resource-sensitive LLM usage and sustainable mobile and web design.

References

- [An17] Angela Blackwell: The Curb-Cut Effect, 2017, https://ssir.org/articles/entry/the_curb_cut_effect, accessed: 06/05/2026.
- [An23] Andersen, Michael: Sustainable Web Design in 20 Lessons. Michael Andersen, 2023.
- [Ax26] Axenbeck, J. et al.: Between 2010 and 2021, global emissions from digital technologies were largely obscured in greenhouse gas emission accounting standards. *Communications Sustainability* 1 (1), p. 25, 2026, <https://doi.org/10.1038/s44458-025-00022-6>.
- [Ba23] Barker, Michelle: Introduction to web sustainability, 2023, <https://developer.mozilla.org/en-US/blog/introduction-to-web-sustainability/>, accessed: 06/05/2026.

- [Ca23] Capgemini: A World in Balance 2023: Heightened sustainability awareness yet lagging actions, 2023, https://www.capgemini.com/wp-content/uploads/2023/11/CRI_World-in-balance-1.pdf, accessed: 06/05/2026.
- [Co24] Cottier, Ben and Rahman, Robi and Fattorini, Loredana and Maslej, Nestor and Besiroglu, Tamay and Owen, David: The rising costs of training frontier AI models. arXiv preprint arXiv:2405.21015, 2024.
- [Da20] Danny van Kooten: CO2 emissions on the web, 2020, <https://www.dannyvankooten.com/blog/2020/website-carbon-emissions/>, accessed: 06/05/2026.
- [Gr18] Gröger, J. et al.: Entwicklung und Anwendung von Bewertungsgrundlagen für ressourceneffiziente Software unter Berücksichtigung bestehender Methodik: Abschlussbericht. Umweltbundesamt, 2018.
- [Gr21] Greenwood, Tom: Sustainable web design. A Book Apart, 2021.
- [Gu24] Guldner, A. et al.: Development and evaluation of a reference measurement model for assessing the resource and energy efficiency of software products and components—Green Software Measurement Model (GSMM). *Future Generation Computer Systems* 155, pp. 402–418, 2024.
- [Ke18] Kern, E. et al.: Sustainable software products—Towards assessment criteria for resource and energy efficiency. *Future Generation Computer Systems* 86, pp. 199–210, 2018.
- [Kh20] Khalifeh, A. et al.: Incorporating sustainability into software projects: a conceptual framework. *International Journal of Managing Projects in Business* 13 (6), pp. 1339–1361, 2020.
- [Na11] Naumann, S. et al.: The GREENSOFT Model: A reference model for green and sustainable software and its engineering. *Sustainable Computing: Informatics and Systems* 1 (4), pp. 294–304, 2011.
- [PKM10] Petter, S.; Khazanchi, D.; Murphy, J. D.: A design science based evaluation framework for patterns. *ACM SIGMIS Database: the DATABASE for Advances in Information Systems* 41 (3), pp. 9–26, 2010.
- [Sc26] Schuster, Gerrit: nachhaltiges-webdesign.jetzt, 2026, <https://nachhaltiges-webdesign.jetzt>, accessed: 06/05/2026.
- [St25] Struk, S. et al.: Pattern-Based Approaches for Mixed Reality in Business Contexts: A Structured Literature Review. In: *International Conference on Information Systems (ICIS)*. 2025.
- [SW22] Steimle, T.; Wallach, D.: Collaborative UX design: lean UX und design thinking: Teambasierte Entwicklung menschenzentrierter Produkte. dpunkt.verlag, 2022.
- [SWC25] Solovyeva, L.; Weidmann, S.; Castor, F.: AI-powered, but power-hungry? Energy efficiency of LLM-generated code. In: *2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge)*. IEEE, pp. 49–60, 2025.
- [To21] Tomitsch, M. et al.: Non-human personas: Including nature in the participatory design of smart cities. *Interaction Design and Architecture (s)* 50 (50), pp. 102–130, 2021.
- [W326] W3C Working Group: Web Sustainability Guidelines (WSG), 2026, <https://w3c.github.io/sustainableweb-wsg/#full-document>, accessed: 06/05/2026.
- [WCS17] Wallach, D.; Conrad, J.; Steimle, T.: The UX metrics table: A missing artifact. In: *International conference of design, user experience, and usability*. Springer, pp. 507–517, 2017.